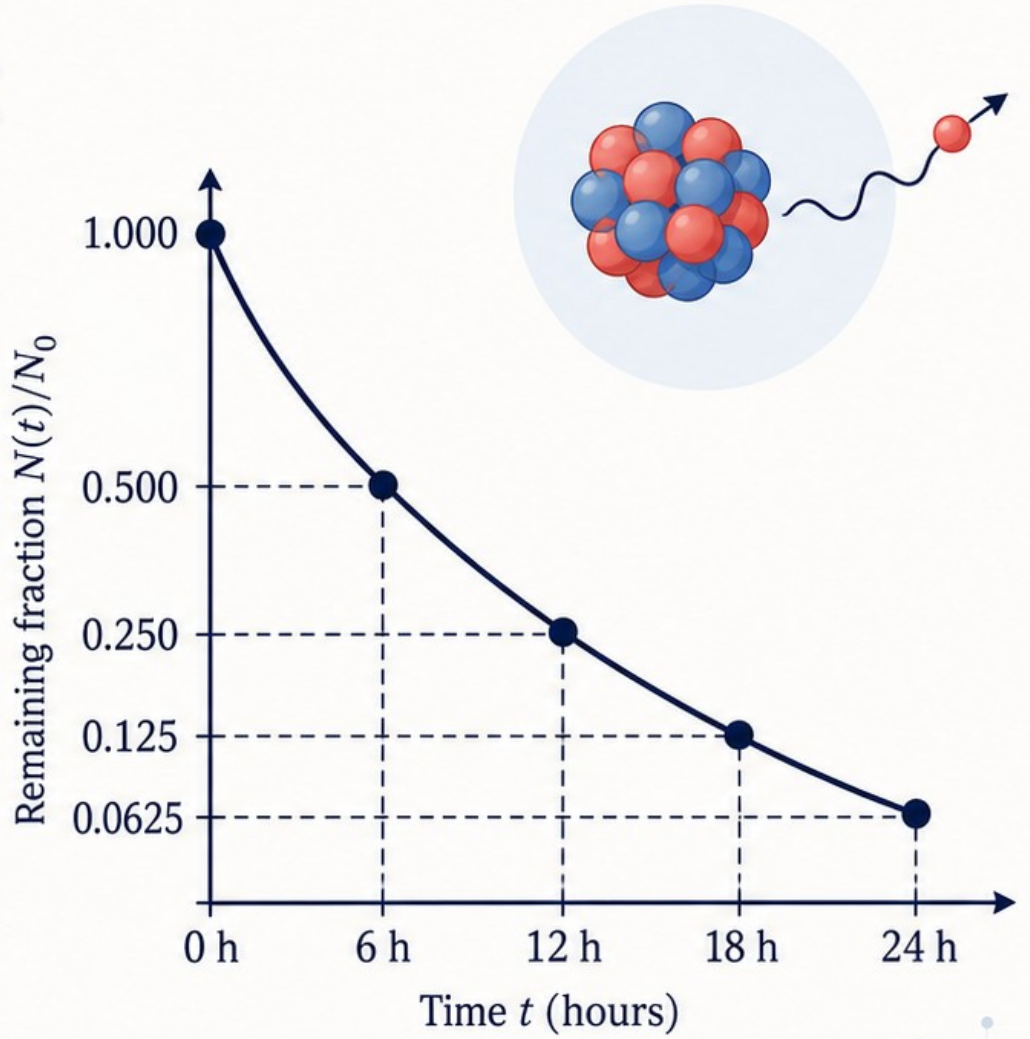
From a physics model to a computational experiment

Week 1 | Programming fundamentals through radioactive decay

A sample has a half-life of 6.0 h.

How should we simulate one day — and decide whether the answer deserves trust?

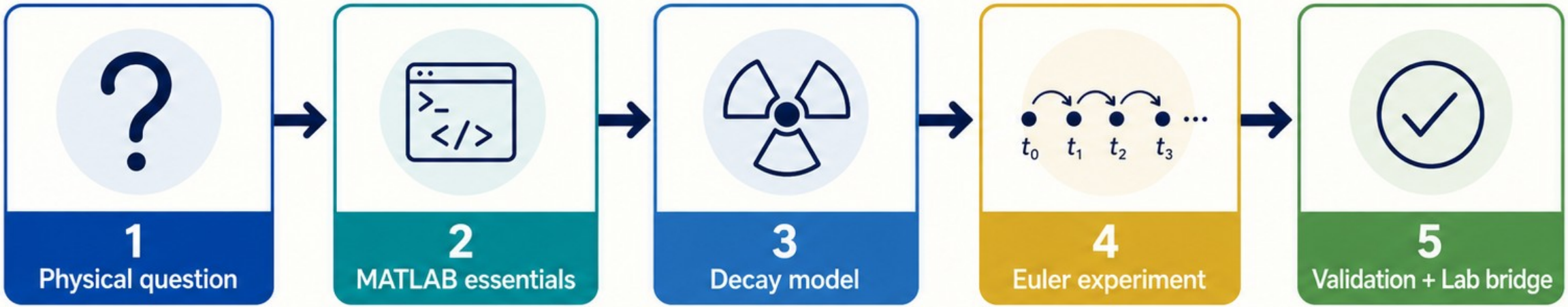
model | units | algorithm | evidence



Today's arc: ask → compute → test

A 120-minute laboratory-style lecture

120 minutes



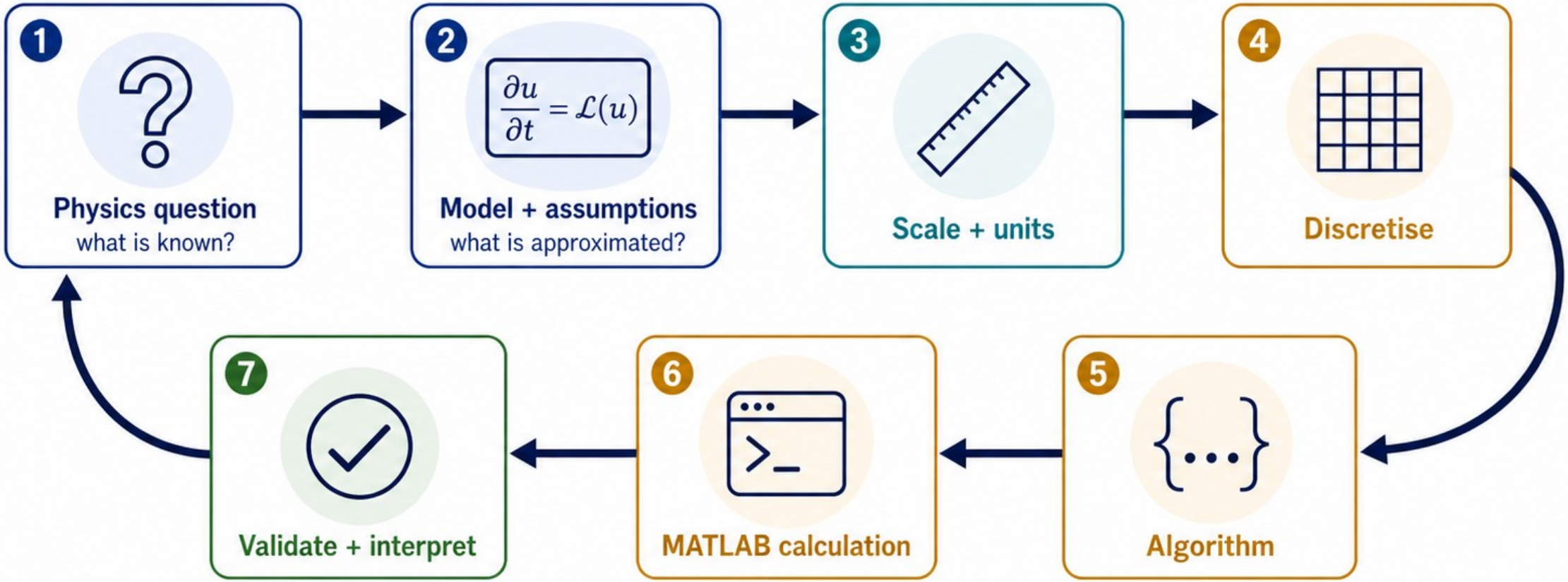
Learning outcomes

- Map physical quantities to variables and units
- Implement one readable time-step loop
- Compare with a reference and justify dt
- Record evidence so another person can rerun it

Course roadmap: Week 1 foundations → later numerical methods

Computation is a controlled experiment on a model

Every arrow carries a decision that should remain visible.



A graph alone is not evidence.

Four MATLAB views, one repeatable experiment

Know where to try, save, inspect, and rerun your work.

Command Window

Try one expression and inspect its answer.

Editor / Live Editor

Save the sequence of choices: text + code + output.

Workspace

Inspect variables currently in memory — useful, but easy to treat as hidden input.

Current Folder

Keep the script and related files together.

Use the Editor / Live Editor for work you intend to rerun.

Readable names make units and assumptions visible

The syntax is small; the naming discipline is the habit.

```
N0 = 1000;                % initial population, nuclei

T_half_h = 6.0;           % half-life, h

lambda_per_h = log(2)/T_half_h;  % decay constant, 1/h
```

Assignment

Use = to store a value in a named variable.

Operators

Use arithmetic to translate an expression.

Comments

Use % to explain a choice to the next reader.

Unit naming

A suffix such as _h or _m2 keeps dimensions visible.

Unit check: `lambda_per_h` × `dt_h` must be dimensionless.

A unit conversion is already a small computational model

Lab 01 warm-up: state the givens, translate the units, inspect the magnitude.

Given: 2 gallons + 4 pints

8 pints = 1 gallon

1.76 pints = 1 litre

1 inputs

2 gal

+

4 pt

2 total pints

```
totalPints
= 2 × 8 + 4
= 20
```

3 litres

```
volumeLitres
= 20 / 1.76
≈ 11.364 L
```

≈ 11.364 L

```
gallons = 2;
pints = 4;
pintsPerGallon = 8;
pintsPerLitre = 1.76;

volumeLitres =
(gallons*pintsPerGallon + pints)/
pintsPerLitre;
```

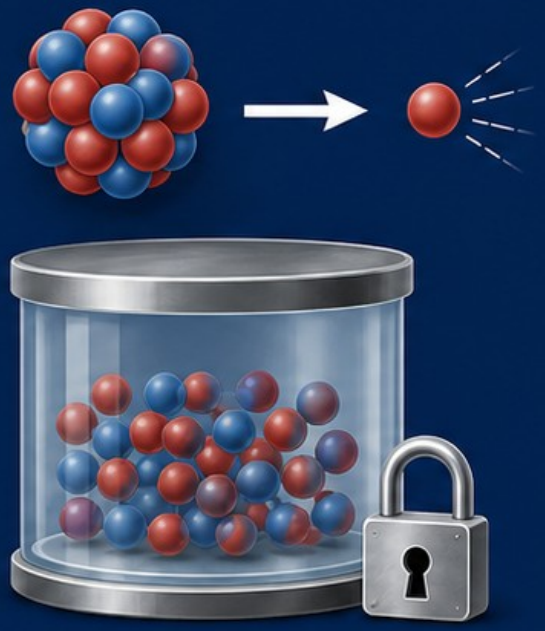
MATLAB habits:
name inputs | show the conversion | check plausibility

Optional Lab 01 bridge: quadratic warm-up with $a = 2$, $b = -10$, $c = 12$

Start with a prediction, then write the model

A closed sample with a constant half-life gives a controlled physical question.

Model



$$\frac{dN}{dt} = -\lambda N$$

$$N(0) = N_0$$

Question

A sample has half-life 6.0 h.
How many undecayed nuclei remain after 24 h?

Assumptions

- constant lambda
- closed sample
- same decay probability per unit time

Predict

- After 6.0 h: 0.5 of N_0
- After 24 h: 0.0625 of N_0

A physical population should not become negative.

Map physics symbols to readable MATLAB names

A parameter map prevents notation and units from drifting during coding.

Physics	MATLAB name	meaning / unit
N_0	`N0`	initial population / nuclei
$T_{1/2}$	`T_half_h`	half-life / h
$lambda$	`lambda_per_h`	decay constant / 1/h
t_{max}	`tMax_h`	final time / h
dt	`dt_h`	numerical step / h

Approved values

T_half_h = 6.0 h

tMax_h = 24 h

lambda_per_h = ln(2)/6.0 ≈ 0.1155 1/h

Dimension check:

lambda_per_h × dt_h
is dimensionless.

The exact solution is a reference, not a shortcut

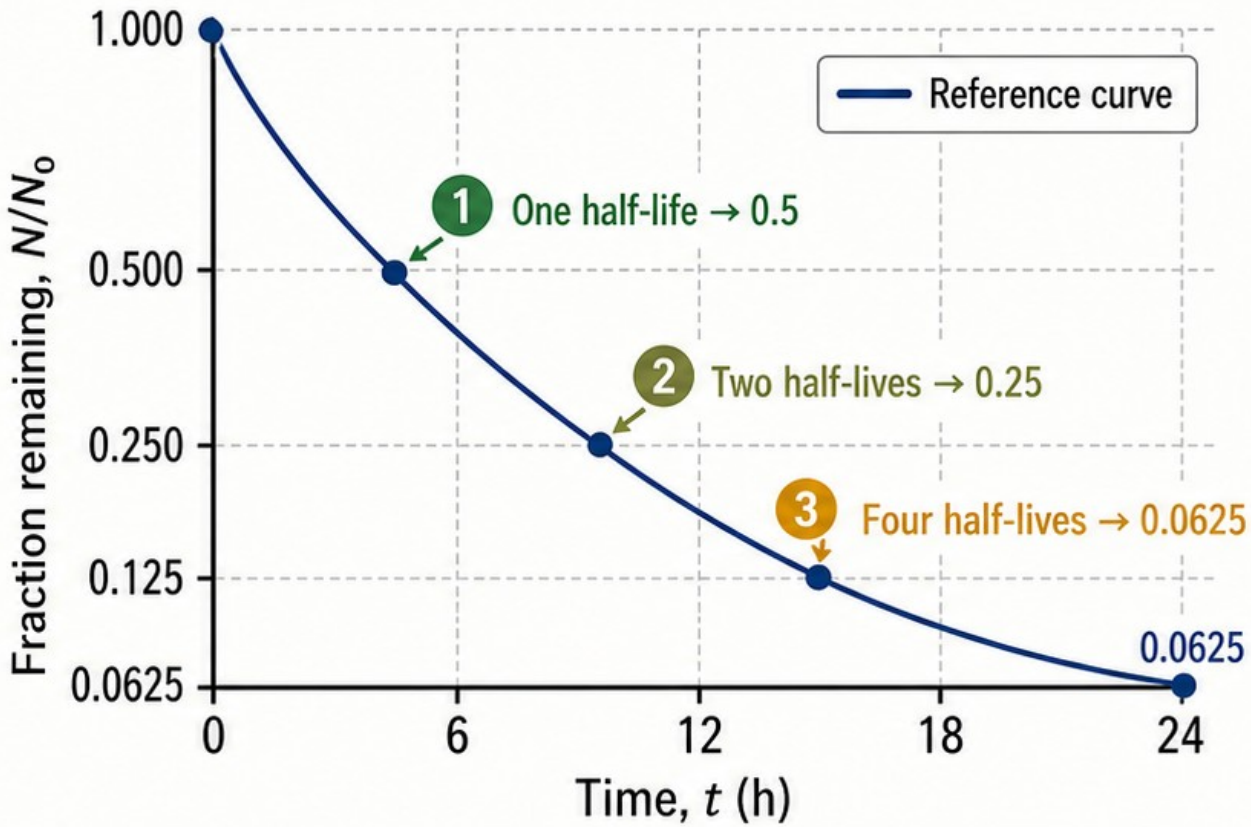
Use the benchmark to make approximation error visible.

$$N_{\text{exact}}(t) = N_0 \exp(-\lambda_{\text{per\_h}}\, t)$$

$$\lambda_{\text{per\_h}} = \ln(2)/6.0 \approx 0.1155\ 1/\text{h}$$

At $t = 24$ h: $N_{\text{exact}}/N_0 = 0.0625$

1. One half-life → 0.5
2. Two half-lives → 0.25
3. Four half-lives → 0.0625



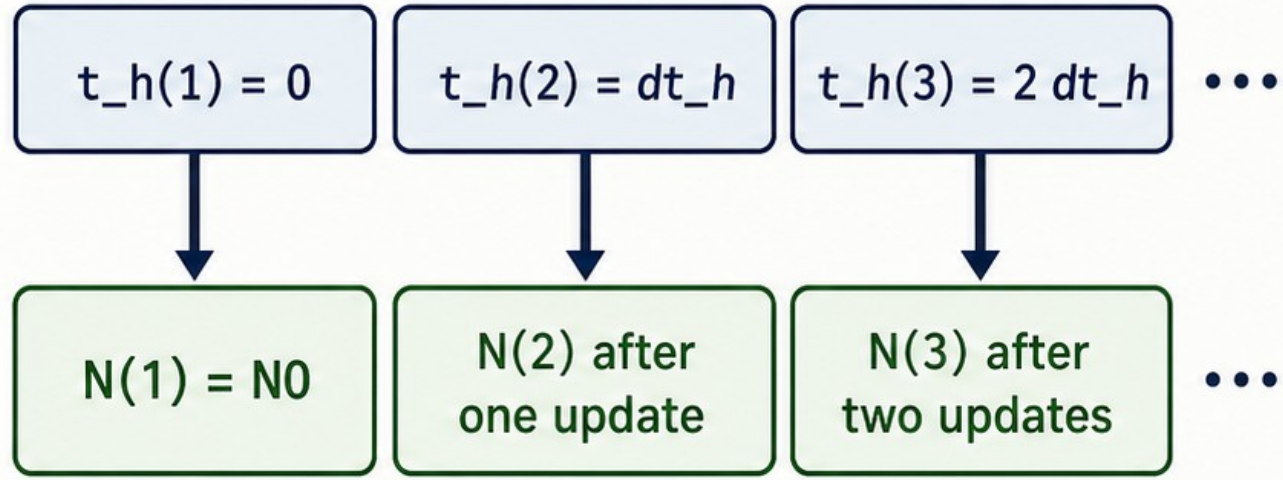
? Prediction before computing:
what should the numerical curve approach?

**A benchmark does not remove the need for numerical work;
it gives the work something to answer to.**

1 Build the time array before the loop

One time value must align with one stored population value.

```
1  t_h = 0:dt_h:tMax_h;
2  N = zeros(size(t_h));
3  N(1) = N0;
```



Preallocation

`zeros(size(t_h))` reserves the storage plan.

Initial condition

`N(1)` is N0 because MATLAB indexing starts at 1.

Alignment

- `t_h(1) = 0` ↔ `N(1) = N0`
- `t_h(2) = dt_h` ↔ `N(2)` after one update
- `t_h(3) = 2 dt_h` ↔ `N(3)` after two updates

Common trap: `N(0)` is not a MATLAB index.

Turn the derivative into a next-value rule

Forward Euler keeps the model but approximates the time derivative.

1 **Start with the model:**

$$\frac{dN}{dt} = -\lambda N$$

2 **Approximate the slope:**

$$\frac{N_{n+1} - N_n}{dt} \approx -\lambda N_n$$

3 **Rearrange:**

$$N_{n+1} = N_n(1 - \lambda dt)$$

Approximation

This is not the exact physics; it is a discrete method for carrying the model forward.

Smaller dt

usually improves the approximation here

More steps

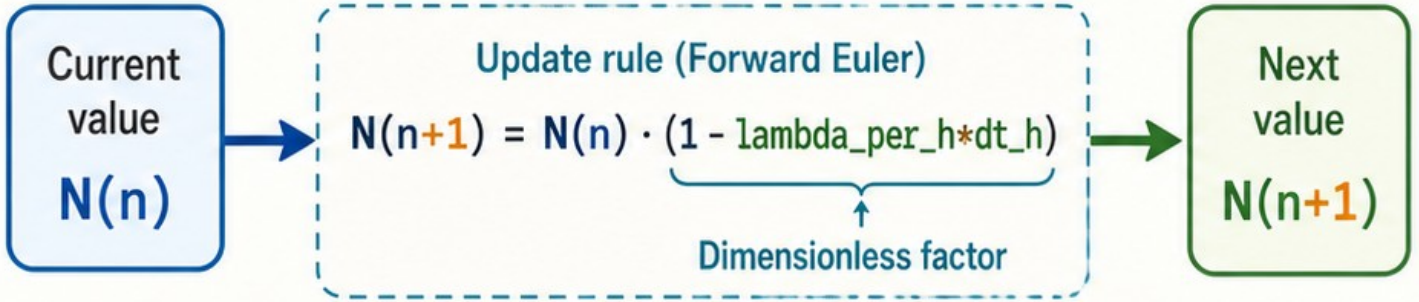
increase the computational cost

Warning: if $\lambda dt > 1$, the factor can become negative.

The loop is the algorithm written out

Read it as current value → update rule → next value.

```
for n = 1:numel(t_h)-1
    N(n+1) = N(n)*(1 - lambda_per_h*dt_h);
end
```



Checkpoint: what does one pass through the loop represent physically?

1 Read the current value `N(n)`

`N(n)` → current value

2 Multiply by the dimensionless factor `1 - lambda_per_h*dt_h`

`N(n)` × `1 - lambda_per_h*dt_h`

3 Store the new value in `N(n+1)`

`N(n+1)` → next value

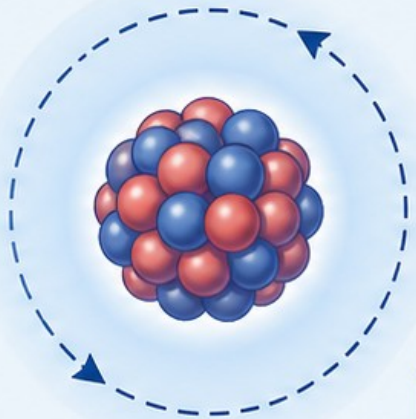
Common mistakes

- Overwrite `N(n)` instead of writing `N(n+1)`
- Loop to `numel(t_h)` and write past the end
- Forget $N(1) = N_0$ before the loop

Hold the physics fixed; vary one numerical choice

A fair timestep experiment changes resolution while keeping the model the same.

Physical model



$N_0 = 1000$

$T_{half\_h} = 6.0\ h$

$t_{Max\_h} = 24\ h$

$lambda\_per\_h = \ln(2)/6.0$

One variable changed

All else held fixed

Numerical resolution

dtValues_h = [2.0 1.0 0.5 0.1]

2.0 h	1.0 h	0.5 h	0.1 h

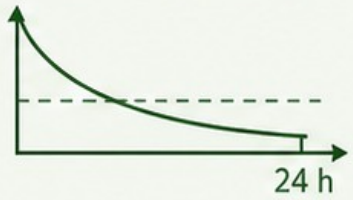
Same update rule; different number of steps

Step count over 0 to 24 h

12 steps	24 steps	48 steps	240 steps
dt = 2.0 h	dt = 1.0 h	dt = 0.5 h	dt = 0.1 h

Record

- Euler fraction at 24 h



- relative error against exact

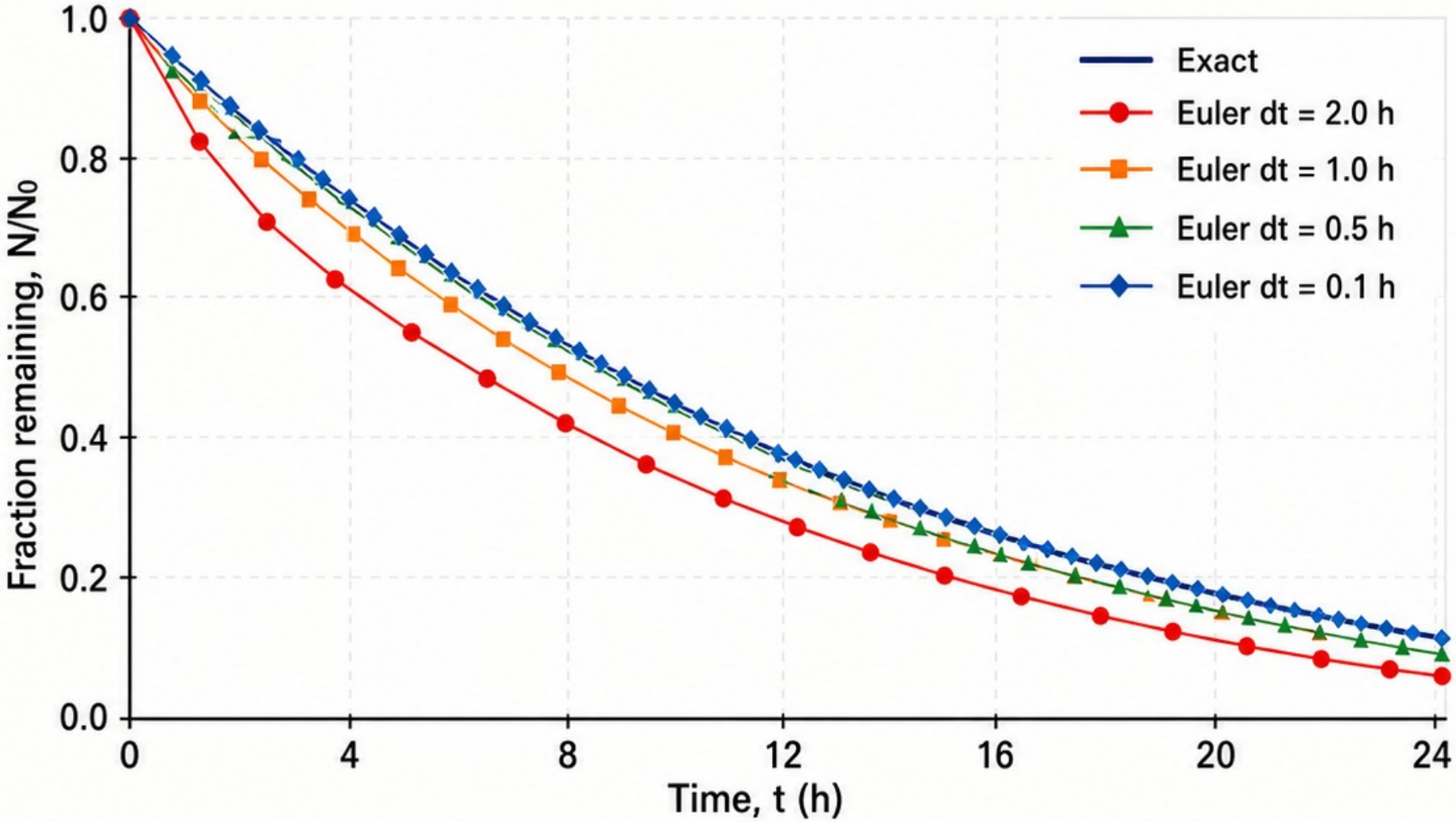
$$\left|\frac{N_{num}(24\ h) - N_{exact}(24\ h)}{N_{exact}(24\ h)}\right| \times 100\%$$

- step count

Cost is part of the result.

Coarse and fine Euler curves separate from the reference

The visual comparison shows why resolution is a modelling decision.



Read the shape

The visual comparison shows why resolution is a modelling decision.

Coarse

Coarse Euler steps fall below the exact curve at 24 h.

Fine

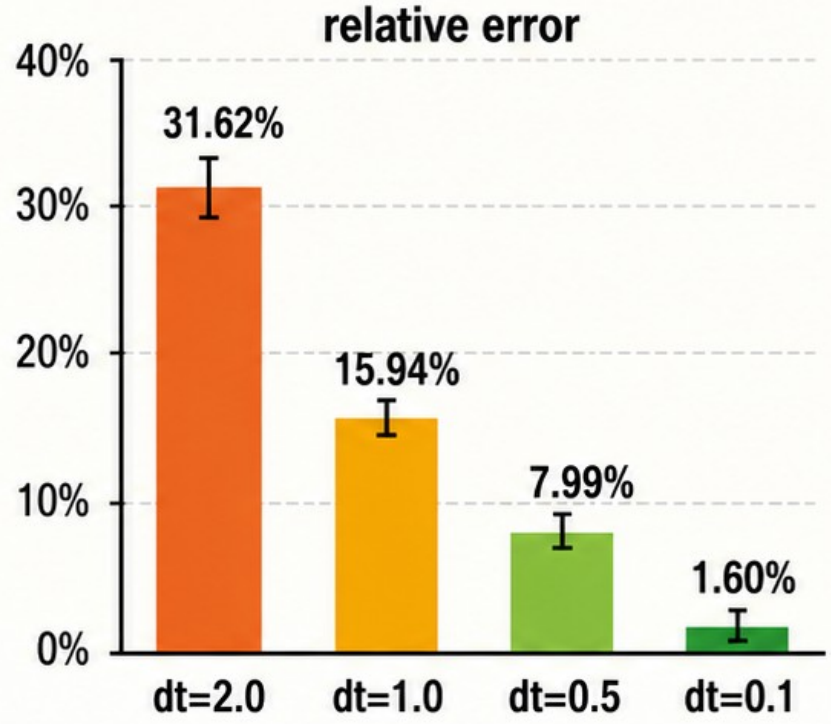
Fine steps track the reference more closely.

Question: which curve would you report first — and what evidence would you add?

Smaller dt improves the result — at a measurable cost

Approved checkpoints for T_half_h = 6.0 h and tMax_h = 24 h

dt (h)	lambda*dt	Euler N/N0 at 24 h	relative error	steps
2.0	0.2310	0.042735	31.62%	12
1.0	0.1155	0.052536	15.94%	24
0.5	0.0578	0.057505	7.99%	48
0.1	0.0116	0.061499	1.60%	240



Exact reference at 24 h: N/N0 = 0.0625

Trend: smaller dt → lower error, more steps

A numerical result includes accuracy and cost.

Trust comes from a trail of checks

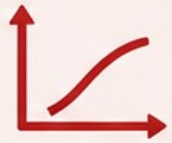
Validation is evidence that the implementation and interpretation agree with a trusted check.

1  **Physics**
Question, variables, assumptions, expected trend

2  **Units**
lambda and t use the same time unit

3  **Algorithm**
Update rule, initial condition, index range

4  **Numerics**
At least two timesteps and a relative error

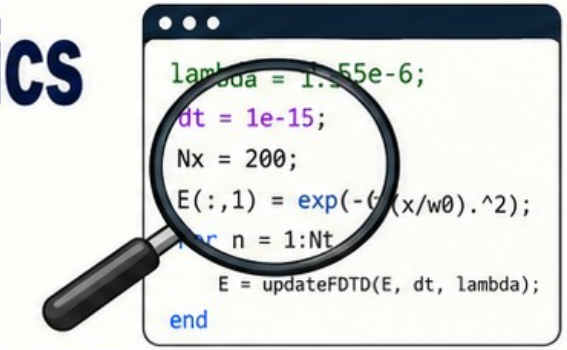
5  **Communication**
Labels, units, legend, and one interpretation

```
assert(abs(N(1)-N0) < eps(N0))

assert(abs(N_exact_fraction(end)-0.0625) < 1e-12)

assert(all(isfinite(N)))

assert(all(N >= 0))
```

 **Warning: lambda*dt > 1 can make the Euler population negative.**

AI can suggest syntax; it cannot certify physics

Responsible AI use means verification literacy, not blind acceptance.

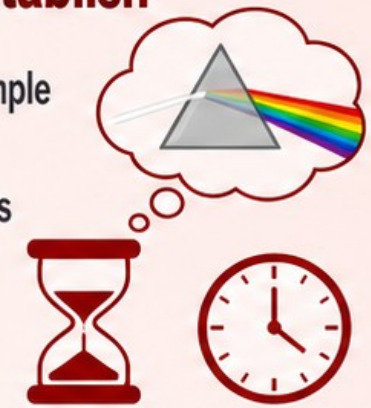


A safe assistance pattern

1. Read the code line by line
2. Check units and limiting cases
3. Compare with N_{exact} and report error
4. Rerun from a fresh MATLAB session
5. State the test you performed

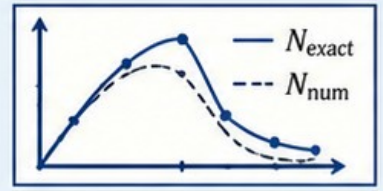
What the assistant cannot establish

- Whether the physical assumptions fit the sample
- Whether lambda and *dt* use compatible units
- Whether the chosen timestep is adequate for the question



Reproducibility record

parameters + units | *dt* + tMax | MATLAB version | validation evidence | labelled figure + caption



Checkpoint: what evidence makes your dt acceptable?

Leave with a model, a method, and a test you can rerun

The Live Script carries today's experiment into the practical.

1 Takeaway 1

A computational result begins with a model and assumptions — not with a MATLAB command.

2 Takeaway 2

dt is part of the numerical method. Smaller is not automatically free.

3 Takeaway 3

Validation and interpretation are required parts of a simulation.

Next in the Week 1 package

1. Run the Live Script from a fresh session.
2. Change one value at a time: T_half_h, dt_h, or N0.
3. Submit the model equation, update line, and validation evidence.

Exit ticket: model + assumption | Euler update | why the timestep is acceptable

model → code → evidence